

Guía de Documentación

Mercado Secundario IFC sobre Avalanche — Hackathon Avalanche LATAM 2026

Esta guía existe para responder dos preguntas: **qué documentar** y **cómo documentarlo** para que el proyecto sea defensible frente al jurado, transferible al equipo, y adoptable por una IFC real (Arkangeles, Bankaool, Play Business) después del hackathon.

1. Por qué documentar

Vas a tener cuatro audiencias distintas leyendo este proyecto. Si escribes un solo documento que intenta servirles a todas, no le sirve a ninguna. Sepáralas.

Audiencia	Pregunta que se hace primero	Documento principal
Jurado del hackathon	¿Resuelve un problema real? ¿Funciona?	Pitch deck + video demo + landing live
Equipo (futuro tú)	¿Cómo levanto esto en mi máquina? ¿Por qué se decidió X?	README + ARCHITECTURE + ADRs
IFC potencial cliente	¿Pasa CNBV? ¿Qué tan caro es operarlo?	Regulatory notes + roadmap + cost model
Auditor de seguridad	¿Dónde están los hooks de compliance? ¿Quién puede freeze wallets?	Smart contract docs + threat model

Regla práctica: **documenta el "por qué" antes que el "qué"**. El código ya muestra qué hace; lo que falta capturar son las decisiones que llevaron a este código y no a otro.

2. Documentos imprescindibles (mínimo viable)

Si sólo escribes seis documentos, escribe estos:

- `README.md` — pitch en 30 segundos + quickstart copy-pasteable
- `ARCHITECTURE.md` — diagrama de capas + decisiones clave
- Pitch deck** (10–12 slides máximo) — para jurados
- `DEMO-SCRIPT.md` (3 min) — guion para presentación en vivo + video
- `docs/regulatory-notes.md` — alineación con CNBV / Ley Fintech
- `.env.example` — todas las variables documentadas con un comentario de cuándo se necesitan

Estos seis te cubren el 80% del valor. Todo lo demás es opcional según el tiempo que tengas.

3. Estructura recomendada del repo

Lo que ya tienes está bien. Ajusta según necesidad:

```
hackathon-avalanche/
├── README.md                ← punto de entrada del repo
├── ARCHITECTURE.md          ← diseño técnico (alto nivel)
├── CLAUDE.md                ← convenciones internas + hard rules
├── CHANGELOG.md             ← qué cambió y cuándo (opcional)
├── docs/
│   ├── GUIA-DOCUMENTACION.md ← este archivo
│   ├── compliance-flow.md    ← flujo KYC + módulos de compliance
│   ├── trading-flow.md       ← matching off-chain + settlement on-chain
│   ├── regulatory-notes.md   ← CNBV / Ley Fintech
│   ├── api-reference.md      ← (a crear) endpoints REST
│   ├── smart-contracts.md    ← (a crear) ABI y responsabilidades
│   ├── runbook.md            ← (a crear) cómo operar en producción
│   └── adr/                  ← Architecture Decision Records
│       ├── 001-erc-3643-vs-erc-1404.md
│       ├── 002-postgres-vs-mongo.md
│       └── 003-mock-blockchain-layer.md
├── pitch/
│   ├── deck.pdf              ← (a crear) slide deck final
│   ├── one-pager.pdf         ← (a crear) PDF de 1 página
│   └── demo-script.md        ← (a crear) guion del video
└── packages/contracts/contracts/
    └── *.sol                 ← NatSpec en cada función pública
```

4. Documentación técnica

4.1 README.md (la cara del proyecto)

Estructura recomendada, en orden:

1. **One-liner** del proyecto (una sola frase, sin jerga)
2. **Hero shot**: GIF o imagen del producto (idealmente del orderbook animado o el dashboard con balances reales)
3. **El problema** — 3-4 líneas, con un dato cuantificado si lo tienes
4. **La solución** — 3-4 líneas
5. **Stack** — tabla compacta
6. **Quickstart** — comandos exactos para levantar todo en local
7. **Arquitectura** — link a `ARCHITECTURE.md` con un mini-diagrama
8. **Roadmap del hackathon** — tabla día por día

9. Equipo + contacto

10. Licencia

Anti-patrón: README de 200 líneas con todo el detalle técnico. Eso va en `docs/`. El README es la *portada*, no el manual.

4.2 ARCHITECTURE.md

Ya lo tienes. Mantén estas secciones:

- **Visión** (1 párrafo, qué resuelve y para quién)
- **Diagrama de alto nivel** (ASCII o imagen)
- **Stack tecnológico** (tabla)
- **Smart contracts** (uno por uno, qué hace cada uno)
- **Flujos clave** (onboarding, emisión, trade, compliance ops)
- **Modelo de datos off-chain** (Prisma schema resumido)
- **APIs principales** (lista de endpoints, link a `api-reference.md` si crece)
- **Infraestructura** (dev, hackathon, producción)
- **Compliance / Regulatorio** (resumen, link a `regulatory-notes.md`)

Cada decisión técnica controversial merece un párrafo de "**por qué esto y no la alternativa**". Ej: "Por qué subnet propia y no C-Chain", "Por qué matching off-chain", "Por qué Postgres y no Mongo".

4.3 API reference (`docs/api-reference.md`)

Por endpoint:

```

### POST /api/orders

**Auth:** required (cookie JWT, role investor/issuer)
**Rate limit:** 30 req/min por wallet

#### Request body
\\\`json
{
  "offeringId": "uuid",
  "side": "buy" | "sell",
  "qty": "decimal-string",
  "price": "decimal-string",
  "expiresAt": "iso-8601",
  "signature": "0x... (EIP-712 sobre OrderPayload)"
}
\\\`

#### Responses
| Status | Body | Cuándo |
|-----|-----|-----|
| 201 | `{ order: OrderResponseDto }` | Orden firmada y persistida |
| 400 | `{ error: { code: "VALIDATION_ERROR", ... } }` | Body inválido o firma inválida |
| 401 | `{ error: { code: "AUTH_REQUIRED" } }` | Sin sesión |
| 429 | `{ error: { code: "RATE_LIMIT" } }` | Sobre el límite |

#### Side effects
- Persiste fila en `orders` (Postgres)
- Empuja a Redis Sorted Set `orderbook:{offeringId}:{side}`
- Emite evento `OrderPosted` al EventBus

```

Hazlo para los 22 endpoints. **No es relleno:** es lo que un integrador (otra IFC) va a leer cuando intente conectarse.

4.4 Smart contracts (docs/smart-contracts.md)

Por contrato:

- **Responsabilidad** (una frase)
- **Storage layout** (variables de estado importantes)
- **Funciones públicas** con NatSpec
- **Eventos emitidos** (el indexer los necesita)
- **Quién puede llamar qué** (matriz de permisos)
- **Invariantes** (cosas que nunca deben pasar)
- **Riesgos conocidos** (reentrancy, integer overflow, etc.)

Ejemplo de tabla de permisos:

Función	owner	issuer	agent	anyone
<code>mint(to, amount)</code>		✓		
<code>freezeWallet(addr)</code>			✓	
<code>forcedTransfer(...)</code>			✓	
<code>transfer(to, amount)</code>				✓ (con compliance)

4.5 Database schema (`docs/database.md`)

- ER diagram (genera con dbdiagram.io desde Prisma, exporta PNG)
- Tabla por modelo: campos, índices, FK, justificación
- Migrations: por qué existió cada migración

4.6 Decisiones (Architecture Decision Records)

Crea `docs/adr/` y un archivo por decisión grande. Formato corto:

```
# ADR-001: Usar ERC-3643 en lugar de ERC-1404

**Estado:** aceptado
**Fecha:** 2026-05-12
**Decisores:** equipo

## Contexto
Necesitábamos un estándar de security tokens que soporte transfer restrictions
basadas en identidad on-chain y permita forced transfer + freeze (requisito CNBV).

## Decisión
Adoptamos ERC-3643 (T-REX) como base.

## Alternativas consideradas
- ERC-1404: más simple pero sin identity registry separada.
- ERC-1400: spec más vieja, menos adopción institucional.

## Consecuencias
+ Identity y compliance modulares, evolucionables sin redeploy del token.
+ Adopción real (Tokeny, Polymath usan T-REX).
- Más contratos = más superficie de auditoría.
- Curva de aprendizaje mayor para devs nuevos.
```

ADRs candidatos para este proyecto: ERC-3643, Postgres vs Mongo, matching off-chain, Supabase + Upstash vs Docker, Core Wallet en grupo separado, capa mock blockchain.

5. Documentación de producto

5.1 User personas

Mínimo tres, una por rol:

- **Inversionista calificado** (Daniela, 34, fondos propios) — qué quiere ver, qué le da miedo, qué lo convierte
- **Operador IFC** (Carlos, 41, ex-banca) — métricas que cuida, cumplimiento que firma
- **Compliance officer** (Laura, 38, abogada fintech) — qué necesita auditable

Una página por persona. Foto stock + bio + frustraciones + objetivos.

5.2 User flows

Diagrama por flujo, no descripción en prosa. Usa Mermaid (renderiza en GitHub):

```
sequenceDiagram
    Inversionista->>Frontend: Sign sell order EIP-712
    Frontend->>Backend: POST /api/orders
    Backend->>Backend: verify signature
    Backend->>Postgres: insert order
    Backend->>Redis: zadd orderbook
    Backend->>EventBus: emit OrderPosted
    Indexer->>Postgres: update view
```

Flujos a documentar: registro/onboarding, KYC, place order, settlement, freeze wallet, forced transfer, distribución de dividendos.

5.3 Roadmap

Una tabla simple, sin fechas exactas pero con horizontes:

Horizonte	Qué
Hackathon	MVP funcional con mocks, demo en vivo, pitch deck
30 días post-hackathon	Deploy real de contratos a Fuji, KYC integrado con Truora
90 días	Sandbox CNBV aplicado, primera oferta real con Arkangeles
6 meses	Subnet propia en AvaCloud, expansión a 2-3 IFCs adicionales

6. Documentación regulatoria (CNBV / Ley Fintech)

Esta sección es la que **diferencia** el proyecto frente a un DEX genérico. No es opcional para un producto regulado.

Por cada artículo o disposición relevante de la Ley Fintech / Disposiciones de Carácter General CNBV, una entrada:

```
### Disposiciones de Carácter General – Art. XX (Audit log)

**Requisito:** Las IFC deben mantener registro inmutable de operaciones.

**Cómo cumplimos:**
- Tabla `audit_log` en Postgres (append-only por convención del servicio).
- Eventos on-chain emitidos por `SecurityToken` (Transfer, ForcedTransfer,
  WalletFrozen) – inmutables por diseño.
- Backup diario de Postgres a almacenamiento WORM.

**Evidencia:** ver `apps/web/src/lib/server/services/audit.service.ts:12-45`.
```

Esto le da al equipo legal de la IFC un mapa exacto de qué satisface qué.

Otros documentos legales útiles: convenio de operación con custodios, términos de uso para inversionistas, política de privacidad alineada con LFPDPPP.

7. Documentación de operaciones (runbook)

Para post-hackathon. Estructura:

- **Cómo levantar el sistema desde cero** (orden exacto de servicios)
- **Variables de entorno** (qué hace cada una, dónde se obtiene)
- **Backups** (qué se respalda, frecuencia, dónde se restaura)
- **Monitoring** (dashboards, alertas, umbrales)
- **Incident response** (¿quién recibe la página? ¿cuáles son los runbooks por incident type?)
- **Smart contract upgrades** (proceso de governance, multisig, timelock)

Mínimo viable hoy: un documento `docs/runbook.md` con los pasos para levantar todo en una máquina nueva, paso a paso.

8. Documentación para el hackathon (audiencia jurado)

8.1 Pitch deck (10–12 slides)

Cada slide responde una sola pregunta:

1. **Hook** — el dolor en una frase
2. **Problema** — cuantificado (TAM, % de inversionistas atrapados, etc.)
3. **Solución** — el diagrama, no el código

4. **Demo screenshot** — los 3 mejores frames
5. **Por qué Avalanche** — razones técnicas concretas
6. **Compliance moat** — el embedded ERC-3643
7. **Arquitectura** — 4 cajas, no 40
8. **Modelo de negocio** — cómo factura
9. **Tracción / status** — qué construyeron
10. **Roadmap** — siguientes pasos
11. **Equipo**
12. **Pedido** — qué necesitan del jurado y partners

8.2 Demo script (3 minutos)

```
0:00-0:20 Setup. Quién eres, problema en una frase.  
0:20-0:40 "Voy a mostrar cómo un inversionista vende su participación  
en 30 segundos."  
0:40-2:30 Demo en vivo. Sin filosofía. Clicks → cambios visibles  
→ transacciones que cierran on-chain.  
2:30-2:50 "Esto, escalado, desbloquea liquidez para 50,000 inversionistas  
atrapados en ofertas vigentes en México."  
2:50-3:00 CTA. URL del repo + URL del demo.
```

Reglas:

- Cada segundo cuenta. Edita sin piedad.
- Si una transacción tarda más de 5s, prepara dos pestañas y haz cut.
- El audio limpio importa más que el video bonito.

8.3 One-pager

Un PDF de UNA página con:

- Logo + nombre del proyecto
- One-liner
- 3 bullets de problema, 3 de solución
- Stack y status
- QR al demo + QR al repo
- Equipo + contacto

9. Cómo escribir buena documentación

Reglas que ahorran horas:

1. **Pirámide invertida.** Lo más importante en el primer párrafo. Un lector con prisa sólo lee la primera pantalla.
2. **Code blocks copy-pasteables.** Si tienen `<placeholder>`, ponlos entre `<>` para que el lector vea claramente qué reemplazar.
3. **Tablas para comparar, listas para enumerar, prosa para explicar.** No mezcles.
4. **Diagrams as code.** Mermaid, ExcaliDraw, dbdiagram.io. Versionable, editable. Evita PNG opacos donde no puedes corregir un typo.
5. **Fechas absolutas.** "Lanzamos en Q3" envejece mal. Escribe "lanzamos en septiembre 2026".
6. **Un comando, una línea.** Si un comando ocupa 3 líneas, parte el setup en pasos.
7. **Documenta el error path tanto como el happy path.** "Si X falla, ver Y."
8. **Mantenla viva.** Doc desactualizado es peor que doc faltante. Bloquéate 30 min cada viernes para revisar.
9. **Lee como usuario nuevo.** Cierra el proyecto, regrésate en una semana, intenta levantarlo siguiendo sólo el README. Anota cada vez que tienes que pensar.

10. Checklist de entrega para el hackathon

Marca cada uno antes de entregar:

- ☐ `README.md` con pitch + quickstart + screenshot del producto
- ☐ `ARCHITECTURE.md` actualizado al estado real
- ☐ Demo en vivo con URL pública (Vercel) y al menos un usuario seed funcionando
- ☐ Video demo de 2-3 minutos, audio limpio, subtítulos
- ☐ Pitch deck en PDF (10-12 slides)
- ☐ One-pager en PDF
- ☐ `docs/regulatory-notes.md` con el mapa CNBV
- ☐ Repo público en GitHub, último commit limpio (no WIP)
- ☐ Tag de release `v0.1-hackathon` apuntando al commit del demo
- ☐ Contacto del equipo visible (email, LinkedIn) en el README
- ☐ `.env.example` completo, sin secrets reales filtrados
- ☐ CI verde (al menos type-check + lint)

11. Anexo: comandos útiles

Generar PDF desde Markdown

```
npx md-to-pdf docs/GUIA-DOCUMENTACION.md
```

Generar diagrama Mermaid

Pega en <https://mermaid.live>, exporta SVG/PNG, súbelo a [docs/img/](#) .

Exportar Prisma schema a ER diagram

```
npx prisma-erd-generator
```

Verificar que todos los enlaces internos del README funcionan

```
npx markdown-link-check README.md
```

Última recomendación: la documentación que **no se va a leer** es peor que la que no se escribió. Sé brutal cortando. Si algo no responde una pregunta concreta de una audiencia concreta, va fuera.